# Python Scheduler Simulator - version 0.3

A Command Line Simulator for CPU Scheduling Policies


- **Author:** Kishore M R
- **Program:** B.Tech Computer Science Engineering
- **Semester:** 4
- **Project Type:** Learning Project


---

## 1.About This Project

This project is a simulator for CPU Scheduling Policies. The CPU is the component of a computer which runs processes according to a caller's instructions.

To do so, it uses some rules to decide the importance of an algorithms usage to a given scenario, based mainly on two things.

- 1.The Existing Process
- 2.The Incoming Process

The type of algorithm used is also determined by:

- 1.Positional View: Who Arrives First?
- 2.Access View: Who Moves First?
- 3.Instance View: Who Stays Longer?

## 2. What is a Policy?

A Policy determines "handling effectiveness" , not order. If it can't generate a baseline efficiency based on its requirement, it is not a Policy, it is a situational aspect.

### What is an Algorithm?

An algorithm determines how a Policy is implemented.It is only used to determine how the policy completes execution.

## 3. Policies Shown in This Project

- First Come First Serve(Non Preemptive) - See FCFS
- Priority Scheduling(Non Preemptive) - See Priority
- Round Robin(Preemptive by default) - See Round Robin

## 4. The Limitations of This Project

This is a simulator.This is not a real scheduler.

## 5. Project Architecture

This project is made in the Python Programming Language and has 3 main components:

- A Process - A task with an identifier
- A Scheduler - An ordering generator
- An Interface - Controls how the user sees details

## 6.FCFS

Process:

- 1.Execute a process from start to finish
- 2.Done

## 7.Priority

A priority is assigned an order based on some external action and is executed in that order

Process:

- 1.Execute a process
- 2.Check priority of next item and execute until all processes are done.
- 3.Exit

## 8. Round Robin

Process:

- 1.A process is trying to run for a time duration which is specified.
- 2.If the process runs completely within usable time -> done else reorder it so that the process can complete its job later.
- 3.Repeat for all processes
- 4.Done

## 9.How Does the Project Work?

This project is purely command line based and uses flags to control result visibility.

### What does the user have to do, you ask?

The user has to pass the first flag listed in Flags Available.Then the user can do the simulation using the listed flags

## 10. Flags

The following flags are available:

- -h : Vist the help CLI utility
- -a (Required,default is Round Robin) : Specify the algorithm to be used.
- -p (Required, defaults are prespecified) : Specify process count
- -t (Required for Round Robin) : Set total algorithm running time
- -q (global flag for Round Robin) : Specify the time for which each process must run atleast once

- -v (Optional) : Show human readable interpretations
- -A (Optional) : Show what the system is doing
- -r (Optional) : Redirect state transitions into user specified file

Note : Atleast one flag is required to see results

# 11.Results of the Simulation:

## Round Robin

If quantum greater than total time, atleast one process never completes

## FCFS

All processes get completed

## Priority

All processes get completed

# 12. Conclusion

Even if the project is a simulator, it clearly demonstrates scheduler policies and shows the limitations of each.